



Kubernetes Cheat Sheet

A Quick Reference Guide for Kubernetes Commands

Basic Commands

- **Get Nodes:** `kubectl get nodes`
- **Get Pods:** `kubectl get pods`
- **Get Services:** `kubectl get services`
- **Describe Pod:** `kubectl describe pod [pod_name]`
- **Logs:** `kubectl logs [pod_name]`

Secrets Management

- **Create Secret from Literal:** `kubectl create secret generic [name] --from-literal=[key]=[value]`
- **Create Secret from File:** `kubectl create secret generic [name] --from-file=[path]`

ConfigMaps

- **Create ConfigMap from Literal:** `kubectl create configmap [name] --from-literal=[key]=[value]`
- **Create ConfigMap from File:** `kubectl create configmap [name] --from-file=[path]`

Deployments and Scaling

- **Create Deployment:** `kubectl create deployment [name] --image=[image]`
- **Scale Deployment:** `kubectl scale deployment [name] --replicas=[num]`

Persistent Volumes

- **View Persistent Volumes:** `kubectl get pv`
- **View Persistent Volume Claims:** `kubectl get pvc`

Jobs and CronJobs

- **Create Job:** `kubectl create job [name] --image=[image]`
- **Create CronJob:** `kubectl create cronjob [name] --image=[image] --schedule="[schedule]"`

Monitoring

- **View Pod Resource Usage:** `kubectl top pod`
- **View Node Resource Usage:** `kubectl top node`

ClusterRole and ConfigMap Commands

ClusterRole Command Examples

```
# Create a cluster role named "pod-reader" that allows user to perform "get", "watch" and "list" on pods
kubectl create clusterrole pod-reader --verb=get,list,watch --resource=pods

# Create a cluster role named "pod-reader" with ResourceName specified
kubectl create clusterrole pod-reader --verb=get --resource=pods --resource-name=readablepod

# Create a cluster role named "foo" with API Group specified
kubectl create clusterrole foo --verb=get,list,watch --resource=rs.extensions

# Create a cluster role named "foo" with SubResource specified
kubectl create clusterrole foo --verb=get,list,watch --resource=pods,pods/status

# Create a cluster role named "foo" with NonResourceURL specified
kubectl create clusterrole foo --verb=get --non-resource-url=/logs/*

# Create a cluster role name "monitoring" with AggregationRule
kubectl create clusterrole monitoring --aggregation-rule=rbac.example.com/monitor
```

ConfigMap Command Examples

```
# Create a ConfigMap named "example-config" from literal values
kubectl create configmap example-config --from-literal=key1=value1 --from-literal=key2=value2

# Create a ConfigMap named "example-config" from a file
kubectl create configmap example-config --from-file=path/to/file

# Create a ConfigMap from a directory, where each file's name is a key
kubectl create configmap example-config --from-file=path/to/directory

# Get ConfigMap details
kubectl get configmap example-config -o yaml
```

Deployment and Secret Generic Commands

Deployment Command Examples

```
# Create a deployment named "nginx-deployment" using the nginx image
kubectl create deployment nginx-deployment --image=nginx

# Scale the deployment "nginx-deployment" to 3 replicas
kubectl scale deployment nginx-deployment --replicas=3

# Update the image for an existing deployment
kubectl set image deployment/nginx-deployment nginx=nginx:1.16.1

# Rollback to the previous deployment
kubectl rollout undo deployment/nginx-deployment
```

Secret Generic Command Examples

```
# Create a generic secret named "db-secret" with specified keys and values
kubectl create secret generic db-secret --from-literal=username=myuser --from-literal=password=mypass

# Create a generic secret from a file
kubectl create secret generic db-secret --from-file=/path/to/secret/file

# View the details of a secret
kubectl get secret db-secret -o yaml

# Decode a secret (Note: the output is base64 encoded)
kubectl get secret db-secret -o jsonpath='{.data.username}' | base64 --decode
```

App Management

Label Command

```
# Partially update a node using a strategic merge patch, specifying the patch as JSON
kubectl patch node k8s-node-1 -p '{"spec":{"unschedulable":true}}'

# Partially update a node using a strategic merge patch, specifying the patch as YAML
kubectl patch node k8s-node-1 -p $'spec:\n unschedulable: true'

# Partially update a node identified by the type and name specified in "node.json" using strategic merge patch
kubectl patch -f node.json -p '{"spec":{"unschedulable":true}}'

# Update a container's image; spec.containers[*].name is required because it's a merge key
kubectl patch pod valid-pod -p '{"spec":{"containers":[{"name":"kubernetes-serve-hostname","image":"new image"}]}}'

#Update a container's image using a JSON patch with positional arrays
kubectl patch pod valid-pod --type='json' -p='[{"op": "replace", "path": "/spec/containers/0/image", "value":"new image"}]'
```

Patch Command

```
# Partially update a node using a strategic merge patch, specifying the patch as JSON
kubectl patch node k8s-node-1 -p '{"spec":{"unschedulable":true}}'

# Partially update a node using a strategic merge patch, specifying the patch as YAML
kubectl patch node k8s-node-1 -p $'spec:\n unschedulable: true'

# Partially update a node identified by the type and name specified in "node.json" using strategic merge patch
kubectl patch -f node.json -p '{"spec":{"unschedulable":true}}'

# Update a container's image; spec.containers[*].name is required because it's a merge key
kubectl patch pod valid-pod -p '{"spec":{"containers":[{"name":"kubernetes-serve-hostname","image":"new image"}]}}'

#Update a container's image using a JSON patch with positional arrays
kubectl patch pod valid-pod --type='json' -p='[{"op": "replace", "path": "/spec/containers/0/image", "value":"new image"}]'
```

Scale Command

```
# Scale a replica set named 'foo' to 3
kubectl scale --replicas=3 rs/foo

# Scale a resource identified by type and name specified in "foo.yaml" to 3
kubectl scale --replicas=3 -f foo.yaml

# If the deployment named mysql's current size is 2, scale mysql to 3
kubectl scale --current-replicas=2 --replicas=3 deployment/mysql

# Scale multiple replication controllers
kubectl scale --replicas=5 rc/foo rc/bar rc/baz

#Scale stateful set named 'web' to 3
kubectl scale --replicas=3 statefulset/web
```

Apply Command

```
# Apply the configuration in pod.json to a pod
kubectl apply -f ./pod.json

# Apply resources from a directory containing kustomization.yaml - e.g. dir/kustomization.yaml
kubectl apply -k dir/

# Apply the JSON passed into stdin to a pod
cat pod.json | kubectl apply -f -

# Note: --prune is still in Alpha # Apply the configuration in manifest.yaml that matches label app=nginx and delete all other resources that are not in the file
and match label app=nginx
kubectl apply --prune -f manifest.yaml -l app=nginx

#Apply the configuration in manifest.yaml and delete all the other config maps that are not in the file
kubectl apply --prune -f manifest.yaml --all --prune-whitelist=core/v1/ConfigMap
```

Env Command

```
# Update deployment 'registry' with a new environment variable
kubectl set env deployment/registry STORAGE_DIR=/local

# List the environment variables defined on a deployment 'sample-build'
kubectl set env deployment/sample-build --list

# List the environment variables defined on all pods
kubectl set env pods --all --list

# Output modified deployment in YAML, and does not alter the object on the server
kubectl set env deployment/sample-build STORAGE_DIR=/data -o yaml

# Update all containers in all replication controllers to have ENV=prod
kubectl set env rc --all ENV=prod

# Import environment from a secret
kubectl set env --from=secret/mysecret deployment/myapp

# Import environment from a config map with a prefix
kubectl set env --from=configmap/myconfigmap --prefix=MYSQL_ deployment/myapp

# Import specific keys from a config map
kubectl set env --keys=my-example-key --from=configmap/myconfigmap deployment/myapp

# Remove the environment variable ENV from container 'c1' in all deployment configs
kubectl set env deployments --all --containers="c1" ENV-

# Remove the environment variable ENV from a deployment definition on disk
# and update the deployment config on the server
kubectl set env -f deploy.json ENV-

# Set some of the local shell environment into a deployment config on the server
env | grep RAILS_ | kubectl set env -e - deployment/registry
```

KUBECTL SETTINGS AND USAGE

api-resources

```
# Print the supported API resources
kubectl api-resources

# Print the supported API resources with more information
kubectl api-resources -o wide

# Print the supported API resources sorted by a column
kubectl api-resources --sort-by=name

# Print the supported namespaced resources
kubectl api-resources --namespaced=true

# Print the supported non-namespaced resources
kubectl api-resources --namespaced=false
# Print the supported API resources with a specific APIGroup
kubectl api-resources --api-group=extensions
```

Config

```
# Display the current-context
kubectl config current-context

# Set only the "client-key" field on the "cluster-admin" # entry, without touching other values
kubectl config set-credentials cluster-admin --client-key=~/.kube/admin.key

#Define new exec auth plugin args for the "cluster-admin" entry
kubectl config set-credentials cluster-admin --exec-arg=arg1 --exec-arg=arg2

# Get the password for the e2e user
kubectl config view -o jsonpath='{.users[?(@.name == "e2e")].user.password}'

# Set basic auth for the "cluster-admin" entry
kubectl config set-credentials cluster-admin --username=admin --password=uXFGyaY0l3lAsIR

#Rename the context 'old-name' to 'new-name' in your kubeconfig file
kubectl config rename-context old-name new-name

#Set only the server field on the e2e cluster entry without touching other values
kubectl config set-cluster e2e --server=https://1.2.3.4

#Embed certificate authority data for the e2e cluster entry
kubectl config set-cluster e2e --embed-certs --certificate-authority=~/.kube/e2e/kubernetes.ca.crt
```

WORKING WITH APPS

Exec Command

```
# Get output from running the 'date' command from pod mypod, using the first container by default
kubectl exec mypod -- date
#Get output from running the 'date' command in ruby-container from pod mypod
kubectl exec mypod -c ruby-container -- date
#Switch to raw terminal mode; sends stdin to 'bash' in ruby-container from pod mypod # and sends stdout/stderr from 'bash' back to the client
kubectl exec mypod -c ruby-container -i -t -- bash -il
#List contents of /usr from the first container of pod mypod and sort by modification time # If the command you want to execute in the pod has any flags in common
(e.g. -i), # you must use two dashes (--) to separate your command's flags/arguments # Also note, do not surround your command and its flags/arguments with
quotes # unless that is how you would execute it normally (i.e., do ls -t /usr, not "ls -t /usr")
kubectl exec mypod -i -t -- ls -t /usr
# Get output from running 'date' command from the first pod of the deployment mydeployment, using the first container by default
kubectl exec deploy/mydeployment -- date
# Get output from running 'date' command from the first pod of the service myservice, using the first container by default
kubectl exec svc/myservice -- date
```

port-forward Command

```
# Listen on ports 5000 and 6000 locally, forwarding data to/from ports 5000 and 6000 in the pod
kubectl port-forward pod/mypod 5000 6000

# Listen on ports 5000 and 6000 locally, forwarding data to/from ports 5000 and 6000 in a pod selected by the deployment
kubectl port-forward deployment/mydeployment 5000 6000

# Listen on port 8443 locally, forwarding to the targetPort of the service's port named "https" in a pod selected by the service
kubectl port-forward service/myservice 8443:https

# Listen on port 8888 locally, forwarding to 5000 in the pod
kubectl port-forward pod/mypod 8888:5000

# Listen on port 8888 on all addresses, forwarding to 5000 in the pod
kubectl port-forward --address 0.0.0.0 pod/mypod 8888:5000

# Listen on port 8888 on localhost and selected IP, forwarding to 5000 in the pod
kubectl port-forward --address localhost,10.19.21.23 pod/mypod 8888:5000

# Listen on a random port locally, forwarding to 5000 in the pod
kubectl port-forward pod/mypod :5000
```


Proxy Command

```
# To proxy all of the Kubernetes API and nothing else
kubectl proxy --api-prefix=/

# To proxy only part of the Kubernetes API and also some static files
# You can get pods info with 'curl localhost:8001/api/v1/pods'
kubectl proxy --www=/my/files --www-prefix=/static/ --api-prefix=/api/

# To proxy the entire Kubernetes API at a different root
# You can get pods info with 'curl localhost:8001/custom/api/v1/pods'
kubectl proxy --api-prefix=/custom/

# Run a proxy to the Kubernetes API server on port 8011, serving static content from ./local/www/
kubectl proxy --port=8011 --www=./local/www/

# Run a proxy to the Kubernetes API server on an arbitrary local port
# The chosen port for the server will be output to stdout
kubectl proxy --port=0

# Run a proxy to the Kubernetes API server, changing the API prefix to k8s-api
# This makes e.g. the pods API available at localhost:8001/k8s-api/v1/pods/
kubectl proxy --api-prefix=/k8s-api
```

Top Command

```
# Show metrics for all nodes
kubectl top node

# Show metrics for a given node
kubectl top node NODE_NAME

# Show metrics for all pods in the default namespace
kubectl top pod

# Show metrics for all pods in the given namespace
kubectl top pod --namespace=NAMESPACE

# Show metrics for a given pod and its containers
kubectl top pod POD_NAME --containers

# Show metrics for the pods defined by label name=myLabel
kubectl top pod -l name=myLabel
```

Logs Command

```
# Return snapshot logs from pod nginx with only one container
kubectl logs nginx

# Return snapshot logs from pod nginx with multiple containers
kubectl logs nginx --all-containers=true

# Return snapshot logs from all containers in pods defined by label app=nginx
kubectl logs -l app=nginx --all-containers=true

# Return snapshot of previous terminated ruby container logs from pod web-1
kubectl logs -p -c ruby web-1

# Begin streaming the logs of the ruby container in pod web-1
kubectl logs -f -c ruby web-1

# Begin streaming logs from all containers in pods defined by label app=nginx
kubectl logs -f -l app=nginx --all-containers=true

# Display only the most recent 20 lines of output in pod nginx
kubectl logs --tail=20 nginx

# Show all logs from pod nginx written in the last hour
kubectl logs --since=1h nginx

# Show logs from a kubelet with an expired serving certificate
kubectl logs --insecure-skip-tls-verify-backend nginx

# Return snapshot logs from first container of a job named hello
kubectl logs job/hello

# Return snapshot logs from container nginx-1 of a deployment named nginx
kubectl logs deployment/nginx -c nginx-1
```